

LAS TRANSACCIONES

ÍNDICE

1. Transacciones distribuidas.
 - 1.1 Protocolos de compromiso.
2. Sistemas remotos de copias de seguridad.
3. Monitores de procesamiento de transacciones.
 - 3.1 Modelo de servidor único.
 - 3.2 Modelo de varios servidores y un solo encaminador.
 - 3.3 Modelo de varios servidores y varios encaminadores.
 - 3.4 Funciones de los monitores de transacciones.
 - 3.5 Coordinación de aplicaciones utilizando los monitores tp.
4. Sistemas de transacciones de alto rendimiento.
 - 4.1 Bases de datos en memoria principal.
 - 4.2 Compromiso en grupo.
5. Transacciones de larga duración.
 - 5.1 Ejecuciones no secuenciables.
6. Transacciones en tiempo real.
- Conclusiones.
- Bibliografía.

1. TRANSACCIONES DISTRIBUIDAS

El acceso a los diferentes elementos de datos en los sistemas distribuidos suele realizarse mediante las transacciones. Hay que tener en cuenta dos tipos de transacciones: las transacciones locales son las que tienen acceso y actualizan datos sólo en una base de datos local; las transacciones globales son las que tienen acceso y actualizan datos en varias bases de datos locales.

Cada emplazamiento tiene su propio gestor de transacciones locales, cuya función es asegurar las propiedades ACID de las transacciones que se ejecutan en ese emplazamiento. Los diferentes gestores de transacciones colaboran para ejecutar las transacciones globales. Para comprender el modo en que se puede implementar un gestor de este tipo se puede definir un modelo abstracto del sistema de transacciones. Cada emplazamiento del sistema contiene dos subsistemas:

- El gestor de transacciones gestiona la ejecución de las transacciones que tienen acceso a datos guardados en un emplazamiento local. Las transacciones de este tipo pueden ser transacciones locales o parte de transacciones globales (transacciones que se ejecutan en varios emplazamientos).
- El coordinador de transacciones coordina la ejecución de las diferentes transacciones (tanto locales como globales) iniciadas en ese emplazamiento.

La estructura de los gestores de transacciones es parecida en muchos aspectos a la utilizada en el caso de los sistemas centralizados. Cada gestor de transacciones es responsable de:

- Conservar un registro histórico con fines de recuperación.
- Participar en un esquema adecuado de control de concurrencia para coordinar la ejecución concurrente de las transacciones en ejecución en ese emplazamiento.

El subsistema coordinador de transacciones no se necesita en los entornos centralizados, dado que las transacciones tienen acceso a los datos en un único emplazamiento. El coordinador de transacciones es responsable de coordinar la ejecución de todas las transacciones iniciadas en ese emplazamiento. Para cada transacción, el coordinador es responsable de:

- Iniciar la ejecución de la transacción.
- Dividir la transacción en una serie de subtransacciones y distribuir estas subtransacciones para su ejecución en los emplazamientos adecuados.
- Coordinar la terminación de la transacción, lo que puede hacer que la transacción se comprometa en todos los emplazamientos o se aborte en todos ellos.

1.1. PROTOCOLOS DE COMPROMISO

Si hay que asegurar la atomicidad, todos los emplazamientos en los que se ejecutó una transacción T deben ponerse de acuerdo sobre el resultado final de la ejecución. T se debe comprometer en todos los emplazamientos o se debe abortar en todos los emplazamientos. Para asegurar esta propiedad, el coordinador de transacciones de T debe ejecutar un protocolo de compromiso.

Entre los protocolos de compromiso más sencillos y más ampliamente utilizados está el protocolo de compromiso de dos fases (C2F). Una alternativa es el protocolo de compromiso de tres fases (C3F), que evita algunos inconvenientes del protocolo C2F pero aumenta la complejidad y la sobrecarga.

2. SISTEMAS REMOTOS DE COPIAS DE SEGURIDAD

Los sistemas tradicionales de procesamiento de transacciones son sistemas centralizados o sistemas cliente-servidor. Esos sistemas son vulnerables frente a desastres ambientales. Hay una necesidad creciente de sistemas de procesamiento de transacciones que ofrezcan una disponibilidad elevada y que puedan funcionar pese a los desastres ambientales.

Se puede obtener una disponibilidad elevada utilizando bases de datos distribuidas en las que los datos estén replicados y las transacciones actualicen todas las copias de los datos. Se puede utilizar el compromiso de dos fases para la sincronización de los nodos. Si un nodo falla, puede continuar el procesamiento de una transacción utilizando otros nodos. Sin embargo, las acciones como el control de la concurrencia, que antes se llevaban a cabo en un solo nodo, ahora necesitan la sincronización de varios nodos con los retrasos consiguientes. Además, el mismo compromiso de dos fases resulta costoso. Por tanto, la productividad de transacciones se ve afectada con este esquema.

Una opción alternativa es realizar el procesamiento de la transacción en un solo nodo, denominado nodo principal, pero tener un nodo remoto copia de seguridad, en el que repliquen todos los datos del nodo principal (nodo secundario). El nodo remoto debe mantenerse sincronizado con el nodo principal, ya que las actualizaciones se realizan en el nodo principal. La sincronización se obtiene enviando todos los registros del registro histórico desde el nodo principal al nodo remoto copia de seguridad. El nodo remoto copia de seguridad debe hallarse físicamente separado del principal para que una catástrofe en el nodo principal no afecte al nodo remoto copia de seguridad.

Cuando falla el nodo principal, el nodo remoto copia de seguridad asume el procesamiento. En primer lugar lleva a cabo la recuperación utilizando su copia (tal vez anticuada) de los datos del nodo principal y los registros del registro histórico recibidos del mismo. En realidad, el nodo remoto copia de seguridad lleva a cabo acciones de recuperación que se hubieran llevado a cabo en el nodo principal cuando éste se hubiera recuperado. Una vez realizada la recuperación, el nodo remoto copia de seguridad comienza a procesar transacciones.

La disponibilidad aumenta mucho en comparación con los sistemas con un solo nodo, dado que el sistema puede recuperarse aunque se pierdan todos los datos de nodo principal. El rendimiento de los sistemas remotos para copias de seguridad es mejor que el de los sistemas distribuidos con compromiso de dos fases.

Varios aspectos que deben abordarse al diseñar sistemas remotos para copias de seguridad son los siguientes:

- **Detección de fallos.** Es importante que el sistema remoto para la copia de seguridad detecte que el nodo principal ha fallado. El fallo de las líneas de comunicación puede hacer creer al nodo remoto copia de seguridad que el nodo principal ha fallado. Para evitar este problema hay que mantener varios enlaces de comunicaciones con modos de fallo independientes entre el nodo principal y el nodo remoto copia de seguridad.
- **Transferencia de control.** Cuando el nodo principal falla, el nodo copia de seguridad asume el procesamiento y se transforma en el nuevo nodo principal. Cuando el nodo principal original se recupera, puede desempeñar el papel de nodo remoto copia de seguridad o volver a asumir el papel de nodo principal. El nodo principal antiguo debe recibir un registro histórico de actualizaciones realizado por el nodo copia de seguridad mientras el nodo principal antiguo estaba fuera de servicio.
- **Tiempo de recuperación.** Si el registro histórico del nodo remoto copia de seguridad se hace grande, la recuperación puede tardar mucho. El nodo remoto copia de seguridad puede procesar de manera periódica los registros rehacer del registro histórico que haya recibido y realizar un control de manera que se puedan borrar las parte más antiguas del registro histórico. Como consecuencia se puede reducir el retraso antes de que el nodo remoto copia de seguridad asuma el control.

Una configuración de relevo en caliente puede hacer la toma del control por el nodo copia de seguridad casi instantáneo. En esta configuración el nodo remoto copia de seguridad procesa los registros rehacer del registro histórico según lleguen, y aplica las actualizaciones de manera local. Tan pronto como se detecta el fallo del nodo principal, el nodo copia de seguridad completa la recuperación retrocediendo las transacciones incompletas y queda preparado para procesar las nuevas.

- **Tiempo de compromiso.** Para asegurar que las actualizaciones de una transacción comprometida sean duraderas, no se debe declarar comprometida una transacción hasta que sus registros del registro histórico hayan alcanzado el nodo copia de seguridad. Este retraso puede dar lugar a una espera más prolongada para comprometer la transacción, y por eso algunos sistemas permiten grados inferiores de durabilidad. Los grados de durabilidad pueden clasificarse de la manera siguiente:

- Uno seguro. Las transacciones se comprometen tan pronto como sus registros de compromiso del registro histórico se escriben en un almacenamiento estable en el nodo local.
- Dos muy seguro. Las transacciones se comprometen tan pronto como sus registros de compromiso del registro histórico se escriben en un almacenamiento estable en el nodo principal y en el nodo copia de seguridad.
- Dos seguro. Este esquema es idéntico al esquema dos muy seguro si tanto el nodo principal como el nodo copia de seguridad están activos. Si sólo está activo el nodo principal, se permite que la transacción se comprometa tan pronto como su registro de compromiso del registro histórico se escriba en un almacenamiento estable en el nodo local.

3. MONITORES DE PROCESAMIENTO DE TRANSACCIONES

Puede que los usuarios necesiten ejecutar actualizaciones en varios de los sistemas como parte de una única transacción. Por tanto, las propiedades de las transacciones relativas a la consistencia y a la recuperación de los fallos son fundamentales. El protocolo de compromiso de dos fases constituye la base del trabajo con transacciones que abarcan varias bases de datos. Los monitores de procesamiento de transacciones (TP) ofrecen apoyo donde se necesitan niveles de software y arquitecturas de software para coordinar la ejecución de dichas tareas.

En un sistema operativo típico de nuestros días, los usuarios inician una sesión y ejecutan uno o varios procesos. Considérese el sistema de reserva de una línea aérea con miles de terminales de reservas activas en cualquier momento dado, todas ellas conectadas con una computadora central. El sistema operativo tiene una sobrecarga de memoria considerable con cada inicio de sesión y esta sobrecarga de memoria sería excesiva si todos los usuarios de las terminales tuvieran que iniciar la sesión en el sistema informático. Además, por motivos de seguridad, no conviene permitir a demasiados usuarios remotos el acceso sin ligaduras al sistema operativo mediante los inicios de sesión. En lugar de tener un inicio de sesión en cada terminal, una opción alternativa es tener un proceso del servidor que se comuniquen con la terminal y trate la autenticación y ejecute las acciones solicitadas por la terminal.

Este modelo presenta varios problemas con respecto a la utilización de la memoria y la velocidad de procesamiento:

- Las necesidades de memoria de cada proceso son elevadas. Aunque se comparta entre todos los procesadores la memoria para el código del programa, cada proceso consume memoria para los datos locales y los descriptores de los archivos abiertos, así como para la sobrecarga del sistema operativo, como las tablas de páginas para trabajar con la memoria virtual.
- El sistema operativo reparte el tiempo de CPU disponible entre los procesos cambiando de uno a otro (multitarea). Cada cambio de contexto de un proceso al siguiente supone una sobre carga de la CPU.

Con este modelo, dadas las cantidades limitadas de memoria y su potencia de procesamiento disponibles en los años setenta y ochenta, hubiera sido imposible atender a más de unas decenas o centenas de usuarios simultáneamente sin quedarse sin recurso de memoria y de CPU.

3.1. MODELO DE SERVIDOR ÚNICO

Para evitar los problemas citados anteriormente, algunos sistemas introdujeron el concepto de monitor de teleprocesamiento. Estos sistemas tenían un único proceso del servidor al que se conectaban todas las terminales remotas (modelo de servidor único). Las terminales remotas envían solicitudes al proceso del servidor, que las ejecuta. Este modelo también se utiliza en entornos cliente-servidor en los que los clientes envían sus solicitudes a un único proceso del servidor. El proceso del servidor trata las tareas, como la autenticación de los usuarios, que normalmente trataría el sistema operativo. Para evitar el bloqueo de los demás clientes cuando se procese una solicitud de gran tamaño de un cliente, el proceso del servidor es multitenhebrado. El proceso del servidor tiene una hebra de control para cada cliente e implementa su propia multitarea con poca sobrecarga. Ejecuta el código en nombre de un cliente durante un período de tiempo, luego guarda el contexto interno y cambia al código de otro cliente. A diferencia de la sobrecarga de la multitarea completa, el coste del cambio entre hebras es reducido. Los sistemas basados en el modelo de servidor único proporcionaban elevadas tasas de transacción con recursos limitados. Sin embargo, presentaban problemas, especialmente cuando varias aplicaciones tenían acceso a la misma base de datos:

- Todas las aplicaciones se ejecutan como un único proceso, no hay protección entre ellas. Un error en una aplicación puede afectar también a todas las demás aplicaciones. Sería más conveniente ejecutar cada aplicación como un proceso independiente.
- Estos sistemas no son adecuados para las bases de datos paralelas o distribuidas, dado que un proceso del servidor no se puede ejecutar simultáneamente en varias computadoras (sin embargo, las hebras concurrentes de los procesos si se pueden soportar en sistemas de memoria compartida con varios procesadores).

3.2. MODELO DE VARIOS SERVIDORES Y UN SOLO ENCAMINADOR

Una manera de resolver los problemas a causa del uso del *modelo de servidor único* es ejecutar varios procesos del servidor de aplicaciones que tengan acceso a una base de datos común y permitir que los clientes se comuniquen con la aplicación mediante un único proceso de comunicación que encamine las solicitudes. Este modelo se denomina *modelo de varios servidores y un solo encaminador*. Este modelo soporta procesos del servidor independientes para varias aplicaciones; además, cada aplicación puede tener un grupo de procesos del servidor, cualquiera de los cuales puede tratar una sesión cliente. Al igual que antes, cada proceso del servidor puede ser multitenhebrado, de manera que pueda tratar varios clientes de manera concurrente. Los servidores de aplicaciones pueden ejecutarse en diferentes nodos de una base de datos paralela o distribuida y el proceso de comunicaciones puede tratar la coordinación entre los procesos.

3.3. MODELO DE VARIOS SERVIDORES Y VARIOS ENCAMINADORES

En vez de uno solo, una arquitectura más general tiene varios procesos para comunicarse con los clientes. Los procesos de comunicaciones con los clientes interactúan con uno o varios procesos de encaminamiento que encaminen sus solicitudes al servidor apropiado. Los monitores TP de última generación tienen una

arquitectura diferente denominada *modelo de varios servidores y varios encaminadores*. Un proceso de control inicia los demás procesos y supervisa su funcionamiento.

3.4. FUNCIONES DE LOS MONITORES DE TRANSACCIONES

- Un monitor TP hace más que pasar mensajes a los servidores de las aplicaciones. Cuando llegan los mensajes, hay que disponerlos en las colas; por tanto, hay un gestor de colas para los mensajes que lleguen. Los mensajes pueden incluso registrarse en un almacenamiento estable para que, una vez recibidos, se procesen independientemente de los fallos del sistema. La gestión de la autorización y de los servidores de aplicaciones (inicio del servidor y encaminamiento de los mensajes a los servidores) son otras funciones de los monitores TP.
- Proporcionan utilidades de registro histórico, recuperación y control de la concurrencia, lo que permite que los servidores implementen directamente las propiedades ACID de las transacciones si es necesario.
- Proporcionan un almacenamiento estable de colas de los mensajes salientes. Por tanto, un servidor puede enviar un mensaje como parte de una transacción; el monitor TP garantiza que el mensaje se entregará sí (y sólo si) la transacción se compromete, proporcionando así las propiedades ACID no sólo para las acciones de la base de datos, sino también para los mensajes enviados fuera de la base de datos.

3.5. COORDINACIÓN DE APLICACIONES UTILIZANDO LOS MONITORES TP

Las aplicaciones de hoy en día suelen tener que interactuar con varias bases de datos. Puede que tengan que comunicarse con usuarios o con otras aplicaciones desde nodos remotos. Por tanto, también tienen que interactuar con los subsistemas de comunicaciones.

Los monitores TP modernos proporcionan soporte para la construcción y la administración de grandes aplicaciones formadas por varios subsistemas, como las bases de datos, los sistemas heredados y los sistemas de comunicaciones. Los monitores TP tratan cada subsistema como un gestor de recursos que proporciona a las transacciones acceso a un conjunto de recursos. La interfaz entre el monitor TP y el gestor de recursos se define mediante un conjunto de primitivas de las transacciones como *begin-transaction* (iniciar transacción), *commit-transaction* (comprometer transacción), *abort-transaction* (abortar transacción) y *prepare-to-commit-transaction* (preparar para comprometer transacción, para el compromiso de dos fases). El gestor de recursos también debe proporcionar otros servicios a la aplicación, como el suministro de datos.

Los monitores TP proporcionan servicios como la gestión de colas persistentes y el transporte de mensajes, que actúan como gestores de recursos al trabajar con transacciones. Los monitores TP pueden coordinar las transacciones entre estos servicios y los sistemas de bases de datos. Por ejemplo, cuando se ejecuta una transacción de actualización de cola se genera un mensaje de salida y la transacción de solicitud se elimina de la cola de solicitudes; el monitor TP asegura que, independientemente de los fallos, ocurrirán todas estas acciones o no ocurrirá ninguna.

En los sistemas cliente-servidor, los clientes suelen interactuar con los servidores mediante un mecanismo de llamada a procedimientos remotos (*Remote-Procedure-Call*, *RPC*), en donde los clientes realizan llamadas a los procedimientos, que se ejecutan

realmente en el servidor y devuelven los resultados a los clientes. En lo que se refiere al código del cliente que realiza la llamada RPC, es como si se realizar una llamada a un procedimiento local.

También se pueden utilizar los monitores TP para administrar sistemas cliente-servidor complejos de varios servidores y gran número de clientes. Cada servidor se comporta como gestor de recursos y se registra en el monitor TP. El monitor TP coordina las actividades, como los controles y los cierres del sistema. Proporciona seguridad y la autenticación de los clientes. Administra los grupos de servidores añadiendo o eliminando servidores sin que le sistema sufra interrupciones. Finalmente, controla el alcance de los fallos. Si falla un servidor, el monitor TP puede detectar el fallo, abortar las transacciones en curso y dar las transacciones a los servidores de otros nodos, abortando también las transacciones incompletas. Cuando se reinicie el nodo con fallo, el monitor TP podrá gobernar directamente la recuperación de sus gestores de recursos.

Los sistemas modernos de bases de datos permiten la réplica de las bases de datos; las actualizaciones realizadas en el nodo principal se ejecutan también de manera automática en las réplicas del mismo. Si el nodo principal falla, uno de los nodos secundarios asume el control del procesamiento. Los monitores TP pueden utilizarse para ocultar los fallos de las bases de datos en esos sistemas replicados; las solicitudes de transacciones se envían al monitor TP, que pasa los mensajes a una de las réplicas de la base de datos. Si falla un nodo, el monitor TP puede encaminar los mensajes de manera transparente a un nodo copia de seguridad y enmascarar el fallo del primer nodo.

4. SISTEMAS DE TRANSACCIONES DE ALTO RENDIMIENTO

Para permitir un índice elevado de procesamiento de transacciones (centenares de miles de transacciones por segundo) hay que utilizar hardware de alto rendimiento y aprovechar el paralelismo. Estas técnicas por si solas son insuficientes para obtener un rendimiento elevado en el procesamiento de transacciones por las razones siguientes:

- La E/S de disco sigue siendo un cuello de botella en las lecturas los compromisos de las transacciones.
- Las transacciones que se ejecutan en paralelo pueden intentar leer o escribir los mismos elementos de datos, lo que da lugar a conflictos que reducen el paralelismo efectivo. Este problema se acentúa con las transacciones de larga duración.

4.1. BASES DE DATOS EN MEMORIA PRINCIPAL

Generalmente, el rendimiento de los sistemas de bases de datos queda limitado por la velocidad a la que los datos se leen del disco y se escriben en él. Se puede reducir el grado de dependencia del disco de una base de datos aumentando el tamaño de la memoria principal. La elevada latencia del disco aumenta el tiempo para el acceso a un elemento de datos y el número de accesos por segundo. Por tanto, para algunas aplicaciones, como las de control de tiempo real, es necesario guardar los datos en la memoria principal para cumplir los requisitos de rendimiento. El tamaño de memoria necesario para la mayoría de estos sistemas no es excepcionalmente grande, aunque hay por lo menos unas cuantas aplicaciones que necesitan que varios gigabytes de datos

estén residentes en la memoria. A medida que aumenta el tamaño de la memoria, se puede esperar que más aplicaciones tengan datos que quepan en la memoria principal.

Las memorias principales de gran tamaño permiten el procesamiento más rápido de las transacciones, dado que los datos están residentes en la memoria. Sin embargo, todavía quedan limitaciones relacionadas con el disco:

- Los registros del registro histórico deben escribirse en un almacenamiento estable antes de comprometer las transacciones. La mejora del rendimiento hecha posible por el mayor tamaño de la memoria principal puede hacer que el procesamiento del registro histórico se convierta en un cuello de botella. Se puede reducir el tiempo de compromiso creando una memoria intermedia estable para el registro histórico en la memoria principal utilizando memoria apoyada por baterías (también denominada RAM no volátil). La sobrecarga impuesta por el registro histórico puede reducirse también mediante la técnica de compromiso en grupo. La productividad (número de transacciones por segundo) sigue limitada por la velocidad de transferencia de datos del disco del registro histórico.
- Hay que seguir escribiendo los bloques de memoria intermedia marcados como modificados por las transacciones comprometidas para que la cantidad de registro histórico que haya que repetir en el momento de la recuperación se reduzca. Si la velocidad de actualización es extremadamente elevada, la velocidad de transferencia de datos del disco puede convertirse en un cuello de botella.
- Si el sistema cae, se pierde toda la memoria principal. En la recuperación el sistema tendrá una memoria intermedia de la base de datos vacía y habrá que introducir los elementos de datos desde el disco cuando se tenga acceso a ellos. Por tanto, incluso después de completar la recuperación, pasa algún tiempo antes de que la base de datos se cargue completamente en la memoria principal y se pueda reanudar el procesamiento de alta velocidad de las transacciones.

Las bases de datos de la memoria principal ofrecen oportunidades para la optimización:

- Dado que la memoria es más cara que el espacio de disco, las estructuras internas de datos de las bases de datos de la memoria principal tienen que diseñarse para reducir las necesidades de espacio. Sin embargo, las estructuras de datos pueden tener punteros que atravesasen varias páginas, a diferencia de las bases de datos de disco, en las que el coste de E/S de atravesar varias páginas sería excesivamente elevado.
- No hace falta colocar las páginas de la memoria intermedia en la memoria antes de que se tenga acceso a los datos, ya que las páginas de la memoria intermedia no se sustituyen nunca.
- Se deben diseñar las técnicas de procesamiento de consultas para minimizar la sobrecarga de espacio, de modo que no se superen los límites de la memoria principal mientras se evalúa una consulta; esa situación daría lugar a que se paginara el área de intercambio y haría más lento el procesamiento de la consulta.
- Una vez eliminado el cuello de botella de E/S, las operaciones de bloqueos pueden transformarse en cuellos de botella. Esos cuellos de botella deben eliminarse mediante mejoras en la implementación de dichas operaciones.

- Los algoritmos de recuperación pueden optimizarse, ya que rara vez hay que escribir las páginas para hacer sitio para otras páginas.

4.2. COMPROMISO EN GRUPO

El procesamiento del compromiso de una transacción T necesita que se escriba en un almacenamiento estable.

Estas operaciones de salida suelen necesitar la salida de bloques que sólo están parcialmente llenos. Para asegurar que se saquen bloques casi llenos se utiliza la técnica de compromiso en grupo. En lugar de intentar comprometer T cuando se complete, el sistema espera hasta que se haya completado varias transacciones. Luego se compromete conjuntamente este grupo de transacciones. Los bloques escritos en almacenamiento estable pueden contener registros de varias transacciones. Eligiendo meticulosamente el tamaño del grupo, el sistema puede asegurar que los bloques estén llenos cuando se escriban en almacenamiento estable. Esta técnica da lugar, en media, a menos operaciones de salida por transacción comprometida.

Aunque el compromiso en grupo reduzca la sobrecarga impuesta por el registro histórico, da lugar a que las transacciones se retrasen en su compromiso hasta que esté preparado para comprometer un grupo de transacciones suficientemente grande. El retraso es aceptable en sistemas de transacciones de alto rendimiento, dado que no pasa mucho tiempo hasta que un grupo de transacciones está preparado para el compromiso.

5. TRANSACCIONES DE LARGA DURACIÓN

El concepto de transacción se desarrolló inicialmente en el contexto de las aplicaciones de procesamiento de datos, en el que la mayor parte de las transacciones son de corta duración y no interactivas. Surgen graves problemas cuando se aplica este concepto a los sistemas de bases de datos que implican interacción humana. Esas transacciones tienen las propiedades clave siguientes:

- Larga duración. Una vez que se produce la interacción humana con una transacción activa, esa transacción se transforma en una transacción de larga duración desde el punto de vista de la computadora, dado que el tiempo de respuesta humano es lento en comparación con la velocidad de la computadora. Además, en las aplicaciones de diseño, la actividad humana que se modela puede abarcar horas, días o períodos más largos. Por tanto, las transacciones pueden ser de larga duración en términos humanos además de en términos de la máquina.
- Muestra de datos no comprometidos. Los datos generadores y mostrados a los usuarios por las transacciones de larga duración no están comprometidos, dado que es posible que la transacción se aborte. Puede que los usuarios y otras transacciones se vean obligados a leer datos no comprometidos. Si varios usuarios colaboran en un diseño, puede que las transacciones de los usuarios necesiten intercambiar los datos antes de comprometer la transacción.
- Subtareas. Una transacción interactiva puede consistir en un conjunto de subtareas iniciadas por el usuario. Puede que el usuario desee abortar una subtarea sin que haga falta abortar toda la transacción.
- Recuperabilidad. No es aceptable abortar una transacción de larga duración por una caída del sistema. Hay que recuperar la transacción activa en un estado que

existiera poco antes de la caída para que se pierda relativamente poco trabajo humano.

- Rendimiento. El buen rendimiento de un sistema de transacciones interactivo se define como tiempo de respuesta breve. Esta definición contrasta con la de los sistemas no interactivos en los que el objeto es una productividad (número de transacciones por segundo) elevado. Los sistemas de productividad elevada hacen un uso eficaz de los recursos del sistema. Sin embargo, en el caso de las transacciones interactivas, el recurso más costoso es el propio usuario.

5.1. EJECUCIONES NO SECUENCIABLES

Los protocolos de control de la concurrencia tienen efectos negativos en las transacciones de larga duración:

- Bloqueo de dos fases. Cuando no se puede conceder un bloqueo, se obliga a la transacción que lo solicita a esperar a que ese elemento de datos se desbloquee. La duración de la espera es proporcional a la duración de la transacción que mantiene el bloqueo.
Si el elemento de datos está bloqueado por una transacción de corta duración, se espera que el tiempo de espera sea corto (salvo en el caso de un interbloqueo o de una carga extraordinaria del sistema). Sin embargo, si el elemento de datos está bloqueado por una transacción de larga duración., la espera será de larga duración. Los tiempos de espera largos provocan tiempos de respuesta largos y mayores posibilidades de interbloqueo.
- Protocolos basados en grafos. Permiten que los bloqueos se liberen antes que en los protocolos de bloqueo de dos fases y evitar el interbloqueo. Sin embargo, imponen una ordenación de los elementos de datos. Las transacciones deben bloquear los elementos de datos de manera consistente con la ordenación. Como consecuencia, puede que cada transacción bloquee más datos de los que realmente necesita. Además, las transacciones deben mantener los bloqueos hasta que no haya ninguna posibilidad de que vuelvan a hacer falta. Por tanto, es probable que se produzcan esperas de bloqueos de larga duración.
- Protocolos basados en marcas temporales. Nunca exigen que una transacción espere, sin embargo, exigen que las transacciones se aborten en determinadas circunstancias. Si se aborta una transacción de larga duración, se pierde una cantidad de trabajo considerable. Para las transacciones no interactivas ese trabajo perdido es un problema de rendimiento. Para las transacciones interactivas también es un problema de satisfacción del usuario. Es muy poco conveniente para el usuario descubrir que se han echado a perder varias horas de trabajo.
- Validación. Al igual que los protocolos basados en marcas temporales, los protocolos de validación hacen cumplir la secuncialidad abortando las transacciones.

Por tanto, el cumplimiento de la secuencialidad da lugar a esperas de larga duración, a abortar las transacciones de larga duración o a ambas cosas.

Surgen nuevas dificultades con el cumplimiento de la secuencialidad cuando se consideran los problemas de la recuperación. Anteriormente se discutió el problema del retroceso en cascada, en el que abortar una transacción puede provocar abortar otras. Este fenómeno no es conveniente, especialmente con las transacciones de larga

duración. Si se utiliza el bloqueo, hay que mantener bloqueos exclusivos hasta el final de la transacción si hay que evitar el retroceso en cascada. El mantenimiento de estos bloqueos exclusivos, sin embargo, aumenta la duración del tiempo de espera de las transacciones.

Por tanto, parece que el cumplimiento de la atomicidad de las transacciones debe provocar una mayor probabilidad de las esperas de larga duración o crear la posibilidad del retroceso en cascada.

6. SISTEMAS DE TRANSACCIONES EN TIEMPO REAL

En algunas aplicaciones las ligaduras incluyen tiempos límite para los cuales la tarea debe estar completada. Algunos ejemplos de esas aplicaciones son al gestión de las fábricas, el control del tráfico y la planificación. Cuando se incluyen tiempos límite, la corrección de la ejecución deja de ser simplemente un problema de consistencia de las bases de datos. Más bien, hay que considerar la cantidad de tiempos límite superados y por el tiempo que hace que se han superado. Los tiempos límite se caracterizan de la manera siguiente:

- Inflexible. La tarea tiene valor nulo si se completa después del tiempo límite.
- Flexible. La tarea tiene un valor que disminuye si se completa después del tiempo límite y se aproxima a cero a medida que aumenta el retraso.

Los sistemas con tiempos límite se denominan sistemas de tiempo real.

La gestión de transacciones en los sistemas de tiempo real debe tener en cuenta los tiempos límite. Si el protocolo de control de la concurrencia determina que una transacción T, debe esperar, puede hacer que T, supere su tiempo límite. En tales casos puede ser preferible expropiar el bloqueo que mantiene la transacción y permitir que actúe T. Sin embargo, hay que utilizar la expropiación con precaución, porque el tiempo perdido por la transacción expropiada (debido al retroceso y al reinicio) puede hacer que supere su tiempo perdido por la transacción expropiada (debido al retroceso ya al reinicio) puede hacer que supere su tiempo límite. Lamentablemente, es difícil determinar en una situación concreta si es preferible el retroceso o la espera.

Una dificultad importante para el trabajo con las ligaduras de tiempo real surge de la varianza del tiempo de ejecución de las transacciones. En el mejor de los casos, todos los accesos a datos hacen referencia a datos de la memoria intermedia de la base de datos. En el peor, cada acceso hace que se escriba en el disco de la página que contiene los datos a los que hay que tener acceso. Dado que los dos o más accesos a disco necesarios en el caso más pesimista tardan varios órdenes de magnitud más que las referencias a la memoria principal necesarias en el caso más optimista, el tiempo de ejecución de las transacciones sólo puede estimarse de forma muy aproximada si los datos residen en el disco. Por tanto, suelen utilizarse las bases de datos en memoria principal si hay que cumplir ligaduras en tiempo real. Sin embargo, aunque los datos estén residentes en memoria principal, aparecen varianzas en el tiempo de ejecución, debido a las esperas de los bloqueos, a abortar transacciones, etc.

En los sistemas de tiempo real, el problema fundamental son los límites de tiempo. Diseñar un sistema de tiempo real implica asegurar que hay suficiente capacidad de procesamiento para no superar los tiempos límite sin exigir demasiados recursos de hardware. Conseguir este objetivo, a pesar de la varianza del tiempo de ejecución resultante de la gestión de las transacciones, sigue siendo un problema aún sin solución.

CONCLUSIONES

Los sistemas han evolucionado desde un entorno centralizado a un entorno cliente – servidor. La compartición de datos entre distintos emplazamientos ha permitido una mejor distribución de la información en las bases de datos, facilitando el acceso a una información común, pero ha causado una mayor complicación a la hora de ejecutar transacciones que requieran el acceso a distintos nodos. Aprovechando las capacidades de la distribución se pueden utilizar sistemas remotos de copia, obteniendo así un nuevo nivel de seguridad y recuperabilidad ante los fallos.

El procesamiento de las transacciones es uno de los puntos problemáticos en la gestión de bases de datos. Por ello han surgido diferentes herramientas y opciones, como los monitores de TP, que proporcionan consistencia y recuperación de los fallos.

Los sistemas de transacciones avanzadas requieren de hardware específico para mejorar el rendimiento al máximo, pero éste continua siendo uno de los problemas actuales, sobretudo con el aumento de tamaño de las bases de datos y la exigibilidad de los SGBD.

En los sistemas con ligaduras de tiempo real, la corrección de una ejecución no sólo implica la consistencia de la BD, sino también el cumplimiento del tiempo límite.

BIBLIOGRAFÍA

- Fundamentos de Bases de Datos (3ª edición) Abraham Silberschatz, Henry F. Korth, S. Sudarshan.
- Información obtenida de artículos en internet.